pyAmpliCol at one loop

Born–virtual interference: milestone status, benchmarks and architecture

Milestone 2026-10-23 · engine revision e734b708 · AMD EPYC 9754 (shared host)

pyAmpliCol computes the interference $2\,\mathrm{Re}\left(B^{\dagger}GL\right)$ of one-loop and Born amplitudes with the UV and R_2 counterterms of the UFO loop model, for every squared coupling order and all three Laurent coefficients, in binary64, double-double (DD) and arbitrary precision (AP). The engine is the recursive *v2* engine: tree currents, loop topologies and loop numerators come from recursions, and no loop Feynman diagram is enumerated (Section 5). The public counterterm route at this push is still site closure, which the user reserves for test oracles; in production the counterterms are plain tree vertices inserted exactly once, without loop matching (addendum J), work in flight. This report is the status of the intermediate push of 2026-10-23 to the `loop` branch. It separates development measurements from the acceptance campaign, which has not run yet.

What `loop` contains at this push (engine e734b708; merges since 1df22af6)

`06fa573b`	WP2.3	**Stability API and precision carriers.** rotation, reduction-direction and rescaling checks in any combination; DD and AP carriers natively
`bda5e256`	WP2.8	**Schedule M in Rust.** merged, primitive-seeded tries; equal to B at FC, NLC and LC
`d456e789`	WP2.9 part	**Tree electroweak follow-up.** four-quark-line W pairing fixed; Dirac–Weyl class refused or pinned
`2a56a0e7`	reference	**NJet seven-gluon reference.** NJet 3.1.2 / 2.1.1, calibration, fixture, Table 5 re-measured
`42c6117d`	WP2.5	**Observables.** colour and spin correlations, resolved helicities, flavour aliases, Ward sources
`45875b87`	WP2.6	**Public v2 engine and plan v4 artifacts.** engine selection, plan v4 artifacts, poles = all | none, per-point estimates

After `45875b87`: requirements addendum J (`f21495ed`, counterterms) and the fix `e734b708` (canonical stage symbols through the central registry). **Already on `loop`:** v2 waves 0-1 with gate S1, dependency re-pin (NinjaRS aa6a6307, oneloopmaster 8ec256fc, Symbolica 9cdc2949), gate G0, tree-engine EW fixes and hygiene, native Rust executor (WP2.1), colour production (WP2.4). **Requirements addendum** F, G, H, J (heavy processes, portable artifacts, generation budget, counterterms). **Native build** `loop-native-v2-slice-45875b87` (digest `2be17627`…), unchanged by the fix.

Performance gates: policy and status

- **Gate.** Faster than MadLoop and at most 1.30× each matched Recola and OpenLoops time, on the **finite part**, for processes from $2 \to 3$ upward. $2 \to 2$ is reported without a gate.
- **Protocol.** Providers in production mode with poles off where they allow it (OpenLoops `truepoles 0`, Recola without IR poles, MadLoop production build); pyAmpliCol finite-only (`poles = "none"`) with stability checks off.
- **Status.** The acceptance campaign (G1) has **not run**. No gate is claimed passed or failed. Provider references for the gate are in Table 3.
- **Seven gluons.** No matched provider timing exists for $gg \to ggggg$ (Table 1): its speed is reported without a gate, with rotation checks; the Ward check that addendum H requires alongside is still pending (Table 8).
- **History.** The timings of 2026-10-18 measure the superseded per-diagram engine and appear only in Section 6.

G1 pending: no gate result at this milestone

In flight (branches, not on `loop`)

- **Counterterms in the recursion** (WP2.2): on `work/v2-slice` since a18b7599, after this push. After the request-scoped lowering $gg \to gggg$ generates in 257–275 s (sites 243–245 s), its second point in 0.54× the sites' time (DESIGN §6.1 there).
- **`ct-simplify`** (running): removes the witness-table admission and loop matching (addendum J); a public `particle_veto` replaces the internal top-free switch.
- **Generation at scale** (WP3.1, `work/v2-generation-scale`): native trees, Born and counterterms, regeneration-free artifacts, SDK v4; its Rust ports (HANDOFF §0.4) are being split into packages.
- **UV/IR poles** (`work/v2-poles-uvir`, addendum I there).
- **G1** and the live report (WP2.7) not started; **electroweak** and complex mass (WP3.2) pending; the **$2 \to 5$ set** on the WP3.1 branch (Section 1).

User decisions of 23 October

- **Counterterms** (addendum J): model R_2 and UV vertices are plain tree vertices inserted exactly once, admitted by coupling orders and a public `particle_veto`; no witness table or loop matching in production (site closure and witnesses: test oracles only).
- **Poles:** `"uv"` = UV poles of the bare loop, the counterterms' UV poles separate; `"ir"` = Catani poles of the renormalised virtual; one finite part.
- **Rust** (HANDOFF §0.4): heavy, contained generation stages move to rusticol (PyO3 API) with oracle equality; new heavy code in Rust.
- Generation may take O(hours) below 150 GB (addendum H); $gg \to ggggg$ speed without a gate, with rotation and Ward checks; no $2 \to 6$ before the $2 \to 5$ plan is complete.
- Binary64 observables: 10^{-12} unless the own estimate $E > 10^{-12}$ (then E, capped at 10^{-9}, plus DD within 10^{-24}).
- Stability-estimate tests: tenfold margin (error $\leq 10\,E$). Development first, proportionate testing; push to `loop` at milestones with this report.

1 Required processes: references and generation

The plan requires the processes of Table 1 (plan §5.7), compared with MadLoop and Recola, and with OpenLoops where a public library exists. A reference provider is run only if its generation completes in about 45 minutes, or in a time comparable to pyAmpliCol's own generation of the process (HANDOFF §0.1, addendum F). Otherwise pyAmpliCol is validated by Ward and gauge identities, Catani poles, LC/NLC/FC and DD/AP consistency, stability checks, lower multiplicities and existing references.

Table 1: **Generation on this host.** Providers: MadLoop graph generation, export and compilation; Recola process generation; OpenLoops library availability (HANDOFF §0.1). pyAmpliCol: v2 generation + plan export (`LoopSlice.generate`, then plan v4 JSON), plan size and peak resident memory, one thread; a development measurement on `work/v2-generation-scale`, not part of the acceptance campaign. Values stand side by side without a ratio.

Process	pyAmpliCol v2, dev.	MadLoop	Recola	OpenLoops	45-minute rule
$gg \to gggg$	358 s + 109 s 185 MB · 5.8 GiB	> 17 h graph generation alone; colour factors ≈ 89 days (MG5 estimate)	> 57 h, unfinished	no public library	stopped 22 Oct; not to be restarted
$u\bar{u} \to gggg$	193 s + 69 s 70 MB · 2.4 GiB	12.2 h (5 793 + 38 040 + 122 s)	2.8 h (10 220 s)	no public library	over 45 min; results exist and are used
$gg \to gggg$	26.7 s + 5.8 s 15 MB · 0.55 GiB	2.4 h (285 + 8 354 + 21 s)	34 min (2 036 s)	prebuilt ppjjjj	MadLoop over 45 min; results exist and are used
$gg \to t\bar{t}e^-e^+g$ EW	not generated	11 min	22 s	pplttj_ew	within 45 min
$gg \to t\bar{t}e^-e^+g$ QCD	10.1 s + 3.6 s 2 MB · 0.2 GiB	2.7 min	29 s	pplttj	within 45 min
$gg \to t\bar{t}gg$	20.9 s + 2.9 s 5 MB · 0.29 GiB	3.6 min	63 s	pptjj	within 45 min
$u\bar{u} \to e^-e^+ggg$	9.9 s + 3.7 s 2.3 MB · 0.22 GiB	2.4 min	16 s	pplljjj	within 45 min
$u\bar{u} \to e^-e^+d\bar{d}g$	9.5 s + 3.4 s 1.1 MB · 0.2 GiB	56 s	8 s	pplljjj	within 45 min
$2 \to 2, 2 \to 3$ controls	—	< 1 min	< 2 s	—	within 45 min

Table 2: **Validation of the required set** (development , `work/v2-generation-scale`; binary64, all three Laurent coefficients, three exact points per process, trees, Born and counterterms from the Python per-point inputs). Deviation metric $|a-b| \,/\, \max(|a|,|b|,|B|\ \alpha_s/2\pi)$. Points that the stability estimate flags in binary64 are resolved in DoubleFloat. ML: MadLoop; OL: OpenLoops.

Process	References	Agreement
$gg \to gggg$	Recola, OL	Recola, OpenLoops, all coefficients: ≤ 8.9×10^{-12} at point 0 and ≤ 4.4×10^{-11} at point 2 (largest: finite part); point 1: 8.4×10^{-9} in binary64 (flagged, estimate 3.0×10^{-6}), 7.1×10^{-11} in DoubleFloat
$gg \to t\bar{t}gg$	ML, Recola, OL	MadLoop, OpenLoops ≤ 2.1×10^{-11}; Recola ≤ 1.0×10^{-10}
$u\bar{u} \to e^-e^+ggg$	ML, Recola, OL	all three providers ≤ 6.9×10^{-13}
$u\bar{u} \to e^-e^+d\bar{d}g$	ML, Recola, OL	all three providers ≤ 8.5×10^{-13}
$gg \to t\bar{t}e^-e^+g$ QCD	ML, Recola, OL	≤ 3.6×10^{-12} at two points; third point 5.7×10^{-9} in binary64 (estimate 1.7×10^{-9}), MadLoop to 1.9×10^{-14} in DoubleFloat
$gg \to t\bar{t}e^-e^+g$ EW	ML, Recola, OL	not started: Feynman-gauge massive vectors, Goldstones, complex-mass scheme, generic EW counterterms (WP3.2)
$u\bar{u} \to gggg$	ML, Recola	MadLoop ≤ 2.6×10^{-12}, Recola ≤ 6.1×10^{-10} (finite ≤ 2.9×10^{-10}), all coefficients, three points
$gg \to gggg$	NJet (no gate)	Born and poles against NJet (§3); finite part waits for the native counterterms

Table 3: **Provider references for the speed gate**, s per point, one pinned core, production mode (finite part; OpenLoops `truepoles 0` with I-operator poles, Recola without IR poles, MadLoop production build). Measured on 2026-10-20 for the design (DESIGN §1.3, Table 1b) and to be re-frozen by G1; inferred provider values of that table are omitted. Threshold: the largest pyAmpliCol finite-only time that would pass, 1.3× the fastest matched Recola/OpenLoops time where it is below MadLoop's.

Process	OpenLoops	Recola	MadLoop	Threshold
$gg \to gggg$	2.92–2.96	not measured	not measured	3.8
$gg \to t\bar{t}gg$	0.487–0.492	2.02–2.05	5.22–5.29	0.63
$u\bar{u} \to e^-e^+ggg$	0.348–0.353	0.246–0.259	1.41	0.32
$u\bar{u} \to e^-e^+d\bar{d}g$	0.053–0.055	0.035	0.143	0.045
$gg \to t\bar{t}e^-e^+g$ QCD	0.425–0.433	0.59–0.67	3.03–3.04	0.55
$gg \to t\bar{t}e^-e^+g$ EW	0.96–1.12	0.95–1.02	6.90–7.09	1.24
$u\bar{u} \to gggg$	no library	not measured	not measured	—
$gg \to gggg$	no library	none (§0.1)	none (§0.1)	no gate

2 Native runtime of the loop stage (development)

No v2 process has an end-to-end timing under the gate protocol yet (native inputs, finite-only, checks off); development timings of the full virtual on the Python-input path exist (Section 3). The native executor evaluates the loop stage (seeds, dressing, NinjaRS reduction with cached oneloopmaster masters, scatter), while trees, Born amplitudes and counterterm scalars are still formed per point in Python and handed over (native lanes for them are WP3.1 work). Table 4 therefore times the native loop stage alone: a development measurement of schedules B and M (WP2.8), not a gate row. The cells stay neutral: they exclude work that a provider time includes.

Table 4: **Native loop stage**, ms per point, binary64, poles off, no stability check, 16 lanes, 16 points at $\sqrt{s} = 1$ TeV, best of 3 (DESIGN §5.6). Excludes trees, Born and counterterm scalars. B: per-family suffix tries (default); M: merged primitive-seeded entries, M/B as the source prints it (from unrounded times) in brackets. Reduction: share of the 8-thread time spent in the serial NinjaRS reduction with cached masters. γ^*: photon exchange only.

Process	Col.	Families	B, 1 thread	M, 1 thread	B, 8 threads	M, 8 threads	Reduction
$u\bar{u} \to d\bar{d}g$	FC	42	0.89	0.89 (1.01)	0.62	0.58 (0.93)	40 %
$u\bar{u} \to ggg$	FC	90	3.43	3.52 (1.02)	1.39	1.33 (0.96)	48 %
$gg \to ggg$	FC	154	13.93	13.63 (0.98)	4.52	4.55 (1.01)	55 %
$u\bar{u} \to e^-e^+gg$ γ^*	FC	69	2.82	2.83 (1.00)	1.43	1.25 (0.87)	48 %
$u\bar{u} \to gggg$	FC	628	62.33	63.12 (1.01)	19.43	19.59 (1.01)	60 %
$gg \to gggg$	FC	1 100	275	248 (0.90)	78.68	72.51 (0.92)	59 %
$gg \to gggg$	LC	550	95.82	76.29 (0.80)	30.04	27.03 (0.90)	60 %
$u\bar{u} \to e^-e^+d\bar{d}g$ γ^*	FC	242	9.74	9.85 (1.01)	5.08	4.97 (0.98)	63 %
$u\bar{u} \to e^-e^+ggg$ γ^*	FC	463	41.47	42.06 (1.01)	15.64	15.68 (1.00)	66 %

Table 5: **Where the native time goes** (WP2.1 executor, 2026-10-22, release build without LTO; DESIGN §5.5), ms per point, binary64, one thread, 16 lanes, schedule B. Dressing: open-loop segments; factors: per-point vertex tensors; reduction: NinjaRS with cached oneloopmaster masters (not clocked separately); finite only and with the reduction-direction re-test. Last column: the Python per-point inputs the native time excludes.

Process	Families	Dressing	Factors	Reduction	Seeds	Scatter	Finite only	With re-test	Python inputs
$u\bar{u} \to d\bar{d}g$	42	0.45	0.21	0.22	0.02	0.28	1.29	1.48	129
$u\bar{u} \to u\bar{u}g$	68	0.86	0.26	0.36	0.03	0.30	1.96	2.33	225
$u\bar{u} \to ggg$	90	2.36	0.32	0.60	0.10	0.26	3.85	4.46	1 356
$gg \to ggg$	154	10.7	0.72	2.32	0.77	0.34	15.3	17.9	22 485
$u\bar{u} \to e^-e^+gg$ γ^*	69	1.86	0.36	0.59	0.01	0.38	3.35	3.92	215

The dressing dominates the native loop stage and the reduction follows ($gg \to ggg$: 10.7 and 2.32 of 15.3 ms). Poles on or off cost the same there; the finite-only saving lies in the counterterm and Catani work that is still in Python. Schedule M gains only on pure-gluon classes, most at leading colour ($gg \to gggg$ LC: 0.80 on one thread); classes with a quark line stay at 1.00 ± 0.02 on one thread (0.85–1.01 at 8 threads over all quark-line classes of DESIGN §5.6), so B stays the default for them. The Python inputs, two to three orders of magnitude above the native loop stage, are the open item: G1 measures end to end once they run natively.

3 Seven gluons: NJet and pyAmpliCol

3.1 NJet: Table 5 of arXiv:1209.0100, re-measured on this host

NJet is the independent reference for $gg \to ggggg$, where MadLoop and Recola cannot generate under the 45-minute rule and OpenLoops has no library. It is massless five-flavour QCD, and it is not one of the gate's matched providers.

Table 6: **NJet, s per point**, double precision, two evaluations each (NJet's scaling test). Paper: Xeon E3-1240 at 3.3 GHz, NJet 1.0. This host: one pinned core, idle at launch, of the shared host; 10 frozen points per channel, mean (spread ≤ 2 %); the multiplier is time ÷ the paper's. De-symmetrised sums equal the standard sums only after phase-space integration; they are timing figures, not point values.

Channel	Table 5	NJet 3.1.2		NJet 2.1.1		Table 5	NJet 3.1.2		NJet 2.1.1	
	standard full-colour sum					de-symmetrised sum				
$gg \to gg$	0.03	0.00295	×0.1	0.00291	×0.1	0.03	no class		no class	
$gg \to ggg$	0.22	defective channel		0.141	×0.6	0.07	defective channel		0.0452	×0.6
$gg \to gggg$	6.19	4.59	×0.7	4.58	×0.7	0.57	0.39	×0.7	0.38	×0.7
$gg \to ggggg$	171.31	142	×0.8	246.7	×1.4	3.07	2.41	×0.8	4.19	×1.4

At seven gluons one NJet 3.1.2 evaluation without the scaling test takes 71.2 s (standard) and 1.22 s (de-symmetrised), at 69 MB; double-double costs 1 990 s per point. NJet 3.1.2's numerical five-gluon channel is 9/2 too large (NJet 2.1.1 is

used there), and at the unrotated seven-gluon points NJet keeps only 4–6 digits in double precision (NaN at 3 of 10), so the reference values are taken at exactly rotated points.

3.2 pyAmpliCol development measurements

These are **development** measurements on the branches named, on the shared host. Generation and the counterterm program ran in one thread; the bare-loop job's thread count was not recorded, so its wall times are not per-core timings (Table 7). They are not part of the acceptance campaign and not a like-for-like speed comparison: the native path below does not yet include counterterms or R_2.

Table 7: ***gg → ggggg* in full colour, development runs.** Wall times and peak resident memory from `/usr/bin/time` for the whole process, including model loading. Generation and the counterterm program are single-threaded (CPU share 101 %, 101 % and 99 %; `RAYON_NUM_THREADS=1` for the counterterm program). The bare-loop job did not record its thread count (the native executor's default is all cores); the whole job, generation and export included, averaged 168 % CPU. Its 1 769 CPU-s (user + system), less one core over generation (471 s) and export with trees (418 s), leave at most 879 CPU-s for the 146 s of evaluations: up to 6.0 busy cores on average and up to 176 CPU-s per point, so the per-core comparison with NJet is undetermined. Counts of the generated slice: 9 414 families, 810 495 skeletons, 197 539 trie nodes, 4 718 tree blocks, 1 080 colour-seed classes on a 720-element Born basis, 4 837 counterterm sites.

Run (branch)	Stages	Wall	Peak memory
Generation and plan, first run `work/v2-generation-scale`, 03:55 UTC	`LoopSlice.generate` 7.4 min (peak 4.2 GiB), plan export 5.4 min, JSON 38.7 s; 949 MB plan	13.6 min	9.3 GiB
Generation and plan, latest run same branch, 10:42 UTC	`LoopSlice.generate` 6.0 min (peak 4.2 GiB), plan export 1.8 min, JSON 9.05 s; 185 MB plan (derived row links)	8.0 min	5.8 GiB
Counterterms in the recursion `work/v2-ct-recursion`	Plain blocks 9.4 min, CT program 53.5 min; 3 070 042 CT currents, 34 781 532 interactions, 847 root sites, no rule left to site closure; script total 63.4 min. Root-closure (bind) terms not measured; run before the request-scoped lowering (ct-recursion DESIGN §6.1)	66.4 min	52.1 GiB
Bare loop + Born, native trees `work/v2-generation-scale`, 5 points	Binary64; per point the point and one rotation (two evaluations); no counterterms, no R_2; thread count not recorded (up to 6.0 cores, ≤ 176 CPU-s per point)	25.5–41.3 s/pt	18.5 GiB whole job

On the same host (Table 6) NJet takes 142 s per point for its standard sum and 2.41 s for its de-symmetrised sum, two evaluations each, on one pinned core, idle at launch, of the shared host. The native path's 25.5–41.3 s per point (also two evaluations) are wall times of a multi-threaded job: its CPU accounting allows up to ≈6.0 busy cores, ≈176 CPU-s per point, during the evaluations, which is above NJet's 142 s. Whether the native bare path is faster than NJet's standard sum per core is therefore undetermined until a one-thread run exists. The comparison is not like-for-like either: counterterms and R_2 are missing on the pyAmpliCol side, its job ran unpinned and multi-threaded on the same shared host, and the de-symmetrised sum is not a point value. Generation fits addendum H's budget (O(hours), below 150 GB). MASTER_PLAN quotes the same two peaks as KiB/10^6 ("9.7 GB", "54.6 GB"); of the run records, the counterterm record prints KiB/10^6 and the generation record GiB.

3.3 Validation without a matched provider

Table 8: **Checks of the seven-leg QCD results**, **development** measurements on `work/v2-generation-scale`, not part of the acceptance campaign, except the Ward row (WP2.5 on `loop`); binary64, deviation metric of Table 2. NJet values: fixture `gg_ggggg_njet.json` (NJet 3.1.2 double-double at exactly rotated points, scaling-test estimates ≤ 1.3×10^{-24}). The bare single pole is compared after subtracting the counterterm pole of its request, as marked in each row: $(\alpha_s/4\pi)B[(n-2)\beta_0^{(5)}+8T_F/3]=(\alpha_s/4\pi)B\cdot119/3$ for loop_sm (five-flavour coupling renormalisation and the top's decoupling term) and $(\alpha_s/4\pi)(n-2)\beta_0^{(5)}B=(\alpha_s/4\pi)B\cdot115/3$ for the top-free request.

Check	Result	Status
$gg\to ggggg$, native bare loop, 5 points: double pole ÷ Born against $-(\alpha_s/2\pi)\cdot7C_A$	≤ 5.4×10^{-12}; rotation estimates 1.6×10^{-10}–2.9×10^{-6}	passed
$gg\to ggggg$ against NJet, native bare loop (loop_sm, 5 points; shift 119/3)	Born ≤ 1×10^{-13}, double pole ≤ 5.4×10^{-12}, single pole 3.5×10^{-12}–8.3×10^{-10} at points 0–3; 7.2×10^{-8} at point 4, which pyAmpliCol's estimate (2.9×10^{-6}) flags	no deviation beyond the estimates
$gg\to ggggg$ top-free (NJet's scheme) against NJet, native bare loop, 3 points; shift 115/3	Born ≤ 4.9×10^{-14}, double pole ≤ 5.4×10^{-12}, single pole 3.4×10^{-9} and 5.2×10^{-9} (estimates 1.4×10^{-8}, 5.0×10^{-8}), 1.4×10^{-11}	finite part needs the native counterterms
Top-free full virtual against NJet at lower multiplicity	$gg\to gg$: ≤ 2.1×10^{-14} at 10 points, full virtual; $gg\to ggg$: ≤ 2.3×10^{-13} at the stable points; the flagged point 3×10^{-14} in DoubleFloat; $gg\to gggg$: 1.0×10^{-11} at point 0; point 1 1.6×10^{-7} in binary64 (estimate 2.7×10^{-6}), 4.7×10^{-15} in DoubleFloat (`test_loop_v2_topfree.py`)	passed
$u\bar{u}\to ggggg$ full virtual against MadLoop and Recola, 3 points	MadLoop ≤ 2.6×10^{-12}, Recola ≤ 6.1×10^{-10} (finite ≤ 2.9×10^{-10}); the providers agree to 6.2×10^{-10}	passed

Check	Result	Status
Ward identity, one gluon's $\varepsilon \to p$, at seven gluons	asymmetric Ward mode of WP2.5 (loop side only, bare + UV + R_2); passes on $gg \to gg$ and $u\bar{u} \to ggg$ (`test_loop_v2_observables.py`)	not yet run at seven gluons (addendum H)

First $gg \to ggggg$ virtual development `work/v2-generation-scale`, not part of the acceptance campaign (full colour, Born α_s^5, loop_sm with top loops, point 0, $\mu = 1$ TeV; Python per-point inputs with counterterms, about 3 h per evaluation on that development path):

	Born	finite	$1/\varepsilon$	$1/\varepsilon^2$
value, GeV^{-6}	3.62326×10^{-6}	-1.30223×10^{-5}	-6.93874×10^{-6}	-1.42896×10^{-6}
rotated copy, rel. diff.		5.9×10^{-9}	2.6×10^{-9}	2.5×10^{-13}
NJet (no top), deviation	3.8×10^{-14}	top loops	1.2×10^{-9}	1.1×10^{-12}

The finite part differs from NJet's by the top-loop contribution (-1.34×10^{-8}, i.e. $-0.197\ |B|\ \alpha_s/2\pi$), which continues the sign and size seen at four to six legs; it is not a test.

4 Capabilities at this milestone

Table 9: **Capability matrix** of the v2 engine on `loop` at e734b708, with branch work marked.

Feature	Status	Boundary
Recursive engine	supported	Trees, families and loop numerators from recursions; no loop diagram is enumerated (FeynKit graphs are test oracles only). Counterterms: see below.
Full, NLC and LC colour	supported	Exact seed tables in N_c; LC/NLC by generated classes and truncated cells. LC is not yet cheaper to generate at seven legs.
Colour and spin correlations	supported	Directed colour histories at LC/NLC/FC, spin sources, resolved helicities, flavour aliases, Ward sources (WP2.5). Seven-leg directed LC/NLC tables need the Rust seed tables (WP3.1).
Per-order partials	supported	Every squared coupling order at tree level and at one loop, mixed Born orders, per-correction loop orders.
Poles at runtime	partial	`poles = "all" | "none"`: finite-only mode with the finite part bitwise unchanged and the poles explicitly absent. `"uv" | "ir"` in progress with the user's semantics of 23 October.
Stability estimates	supported	Rotation checks (any number, 0 = estimate absent), reduction direction, exact rescaling, in any combination; no automatic escalation (WP2.3).
Precision carriers	supported	Binary64, DoubleFloat, arbitrary precision and exact rationals in the native executor; DD/AP point inputs are formed in Python Decimal until WP3.1.
Native executor, schedules	supported	Rust, 16 lanes, threads; schedule B by default, M native (adopted per class at G1), F as oracle. The NinjaRS reduction runs serially.
Plan v4 artifacts and API	partial	Published and loaded through Python, TOML and CLI; relocation to another path tested on the same host and CPU. Loading regenerates the process (the `LoopSlice`) in Python and checks the plan digest, which addendum G forbids for prepared artifacts; WP3.1 removes it. The Rust/C/C++/Fortran SDK refuses v4 until WP3.1.
Counterterms (model UV, R_2)	partial	Site closure is the public route (QCD incl. massive quarks), a form the user reserves for test oracles; counterterms inside the recursion (WP2.2, still with witness-table admission) are on `work/v2-slice` since `a18b7599`, after this push. Production rule (addendum J): plain tree vertices inserted exactly once, admitted by coupling orders and `particle_veto`, no loop matching; `ct-simplify` removes the matching after WP2.2.
Direct R_2	deferred	A bonus by the user's rule; bounded controls only.
Electroweak, complex mass	open	Tree EW lanes fixed, the Dirac–Weyl class refused; loop EW, Goldstones and CMS are WP3.2.
Required set	in progress	On the WP3.1 branch seven of eight generate and six agree with their provider references in all coefficients (Table 2); $gg \to ggggg$ is validated in Born and poles; the EW target is not started.
Performance gates	open	G1 not run; no gate result (front page).
$gg \to 6g$	deferred	No $2 \to 6$ work until the $2 \to 5$ plan is complete (user, 23 October).

5 Architecture of the v2 engine

5.1 The computed quantity

For Born order a, loop order b and squared coupling order $s = a + b$,

$$V_{s,k} = \mathcal{N} \sum_{a+b=s} 2\,\mathrm{Re}\left[B_a^\dagger G_{BL} L_{b,k}\right], \quad k \in \{0,-1,-2\}, \quad V = C_\varepsilon \left(V_0 + \frac{V_{-1}}{\varepsilon} + \frac{V_{-2}}{\varepsilon^2}\right), \quad C_\varepsilon = \frac{(4\pi)^\varepsilon}{\Gamma(1-\varepsilon)},$$

where G_{BL} is the colour metric between Born and loop colour structures and $\mathcal{N}$ holds the initial-state averages and identical-particle factors (MadLoop/BLHA convention; couplings included, nothing divided by the Born). Directed colour and spin correlations replace $2\,\mathrm{Re}(\ldots)$ by the two cross terms $B_A^\dagger G L_B + L_A^\dagger G B_B$.

5.2 Pipeline

For every denominator family f, sector s and observable column o, v2 builds the four-dimensional numerator $N_{f,s,o}(q) = \sum_\alpha c_\alpha q^\alpha$ that NinjaRS reduces: Born-weighted and helicity-summed, produced by three recursions that share all work (Figure 1).

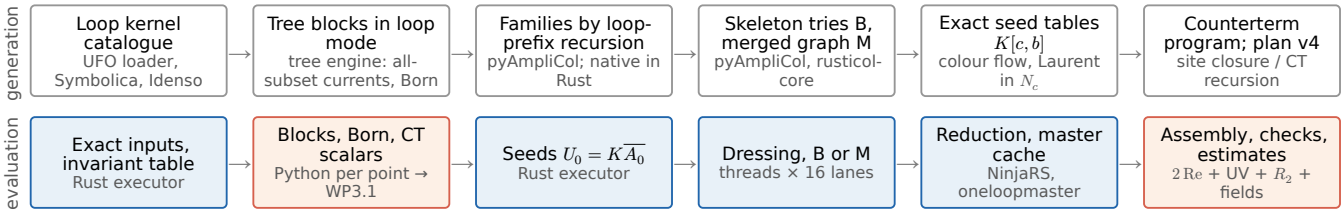


Figure 1: **v2 pipeline.** Upper lane: generation, once per process, into a plan v4 artifact. Lower lane: evaluation per batch of points. Blue: native batched numerics in the Rust executor; orange: still formed in Python per point (native lanes on the WP3.1 branch); white: generation. The owning code or dependency is named under each step.

Trees. The tree engine's colour-ordered Berends–Giele recursion computes, once per point, every off-shell current over every leg subset a loop vertex can absorb (the *blocks*), the Born colour components A_0 and, with WP2.2, every tree with one counterterm vertex. A loop-aware reachability test keeps exactly the currents some family position or root needs. Blocks with transverse sources use the exact transverse Yang–Mills kernels; loop segments, counterterm currents and Ward sources use the full ones.

Families and skeletons. A Recola-type recursion over loop prefixes (ordered leg sets, propagator species and masses) yields the denominator families, keyed by the canonical cyclic sequence of (leg mask, mass) and equal to Recola's tensor-integral identities. Per family it yields the colour-ordered *skeletons*: rings of blocks, each block summing all trees on its leg subset (Figure 2). Skeletons are never stored as a list; they are the leaves of the schedule tries. Only certified structural zeros are dropped (scaleless rings, external wave-function corrections, VEV tadpoles, colour zeros); nothing is restricted to 1PI.

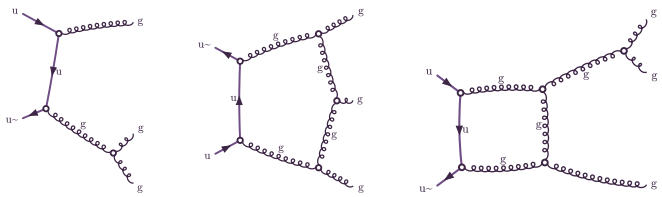


Figure 2: Three graphs of $u\bar{u} \to ggg$ drawn by FeynKit's renderer from its own generation (an oracle, not a production input). In v2 the Born (left) is a closure of blocks; the pentagon (middle) is a ring of five single-leg blocks; the box with a tree attachment (right) is a ring of four blocks, one of them the two-leg current, computed once per point and shared by every skeleton that contains that leg subset.

Loop numerators and schedules. An open-loop recursion dresses skeletons one segment at a time with polynomial-valued objects, $O' = O \cdot Y + \mathrm{raise}\big(O \cdot Z_\mu, q^\mu\big)$, in the NinjaRS monomial layout ($\binom{r+4}{4}$ coefficients at rank r, no rank cap). The seed is the colour-summed Born interference, and each block's helicities are summed as it is attached. One generator feeds three schedules (Table 10).

Table 10: **Three schedules on one generator** (DESIGN §2.2, §5.6).

	F: forward	B: suffix trie (default)	M: merged, primitive-seeded
Born weighting	at closure	at the leaves (one seed per skeleton class)	at the cut, one seed per colour ordering
Shared	prefixes across families	all prefixes with equal family and remainder	prefixes across families and compositions
Colour certificate	none needed	none needed	certified classes: adjoint rings, traces, quark rings
Role	oracle	default; generic fallback	pure-gluon classes; $n \geq 8$ default

Colour. Colour lives only in exact seed tables $K[c,b]$, Laurent polynomials in N_c between seed classes c (the colour vector obtained by closing the cut of a skeleton's colour flow) and Born basis elements b; per point the seeds are $U_0 = \sum_b K[c,b]\overline{A_0[b]}$, with a symmetric-group FFT where certified. LC and NLC differ from full colour only in which classes are generated and how cells are truncated: at LC each colour ordering is seeded by its own Born amplitude and skeletons

below the leading N_c degree are never generated. Certificates (adjoint rings, fundamental traces, quark rings) are proved at loop level, never inherited from the tree level.

Reduction and masters. Families close directly into NinjaRS `TensorNumerator` lanes: one reduction per (family, sector, column), families with equal (propagators, rank) batched as lanes of one call, scalar masters from oneloopmaster cached per event. A second reduction in a relabelled frame reuses the numerators and the cached masters as the cheap stability check.

Counterterms. The model's UV and R_2 vertices, two-point ones included, are ordinary vertices of the tree recursion, and each contribution contains exactly one of them; external field factors are separate sources. Admission is the requested coupling orders, the exactly-once constraint and a public `particle_veto`: a coupling term whose loop particles include a vetoed particle is dropped, and a vetoed particle is removed from every tree vertex, loop line and propagator of the generation, so loops and counterterms stay consistent. Nothing is matched to the loop (user, 23 October; addendum J). At this push the code still admits counterterm sites by a witness table of divergent loop routings with a power-counting completeness check: the public route closes those sites over block currents, and the recursion form of WP2.2 (one counterterm order in the tree recursion, whose $gg \to ggggg$ program leaves no rule to site closure, Table 7) uses the same admission. The package `ct-simplify` removes the witness table and the matching from production and adds `particle_veto`, which replaces the internal top-free switch of Table 8 (then `particle_veto = [t]`); site closure and the witness machinery remain only as test oracles.

Precision, stability and assembly. Every kernel is generic over binary64, DoubleFloat, arbitrary-precision floats and exact rationals. A failed lane fails its point and never branches a kernel. The assembly takes $2\,\mathrm{Re}$ of $i/(16\pi^2)$ times the reduced family sum, plus local UV, R_2 and field terms, per sector and column, with optional rotation, reduction-direction and rescaling estimates and the finite-only mode. The public API selects `loop_engine = "v2"` from TOML, CLI or Python and stores plan v4 as a relocatable PACBIN member (zlib JSON of exact kernel term lists, no compiled code, digest checked at load).

Table 11: **Asymptotic classes per event** for n gluons, full colour (DESIGN §1.4): $F_n \approx (n-1)!\left(\frac{1}{\ln 2}\right)^n/2$ families, 2^n helicities. The required set ($n \le 7$) is far below the crossover of schedule B with the unitarity class ($n \approx 30$–50); schedule M is planned as the default from $n \ge 8$.

Stage or code	Class
Tree blocks	$\Theta(n \cdot n!)$ currents; $\Theta(n! \cdot 2^n)$ current-helicity rows
Dressing, schedule B	$\Theta((n-1)! \cdot 4^n)$ seed injection; trie dressing dominates for $n \le 8$
Dressing, schedule M	$\Theta(n! \cdot 2^n \cdot n^4)$
NinjaRS	$\Theta(F_n)$ reductions, $\Theta(F_n \cdot n^8)$ work, $\Theta(4^n)$ distinct masters after caching
MadLoop (per diagram, per helicity)	$\Theta(D_n \cdot 2^n \cdot n^4)$, D_n super-exponential ($gg \to gggg$: 26 975 loop diagrams)
Unitarity with exact helicity sums (NJet)	$\Theta((n-1)! \cdot 2^n \cdot n^{6\ldots9})$

Table 12: **Ownership of the mathematics.** pyAmpliCol reuses its dependencies for algebra, tensors, graphs and integrals and owns the recursions and the assembly.

Component	Responsibility
ufo_model_loader	Lossless UFO loop-model import: Laurent-valued parameters and couplings, UV/R_2 categories, loop groups
Symbolica	Expressions, coefficient extraction in q, evaluators; exact rationals and arbitrary precision
Spenso, Idenso (Community)	Lorentz and colour tensors behind the kernel catalogue and the colour metrics
FeynKit (Community)	Test oracles: graph and family counts, overall factors, drawings (Figure 2)
NinjaRS, oneloopmaster	Integrand reduction with Laurent expansion; scalar masters in all carriers
pyAmpliCol tree engine	Blocks, Born, coupling orders and colour flow (Python lane; Rust recurrence)
pyAmpliCol loop v2	Families, skeletons, schedules, seed tables, counterterm program, plan v4, native executor, API

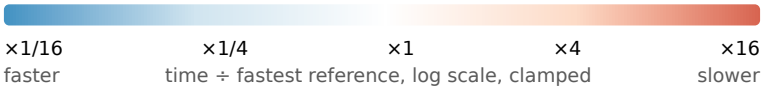
6 History: legacy per-diagram engine, 2026-10-18

History only. These timings measure the superseded per-diagram engine (per-graph coefficient programs, pairwise colour overlaps, whole-process symbolic projectors) through its Python coordinator with exact decimal input, on repeated three-point streams, against providers that kept their protocol settings (actual poles, stability checks and rescue). They are neither the gate

protocol nor the v2 engine, include $2 \to 2$ processes, and are kept for reference without gate counts. Revisions fa599885 and two follow-ups; AMD EPYC 9754 128-Core Processor.

Table 13: **Legacy engine, µs per point** (2026-10-18), binary64, full colour, all three Laurent coefficients; median of five warm calls; multiplier: time ÷ the fastest timed reference of that row, coloured only where MadLoop is timed. Grey rows: generation (pyAmpliCol fresh process to plan; MadLoop export and compilation; Recola generation; OpenLoops setup of a prebuilt library). B: points per call.

Process	B	pyAmpliCol (legacy)		MadLoop		Recola		OpenLoops	
$u\bar{u} \to e^- e^+$ photon only · 1 loop graphs	1	584	×12.5	46.9	fastest	1 880	×40.1	no matched control	
	128	147	×3.2	46.5	fastest	1 780	×38.4	no matched control	
	gen.	2.43 s		198 s		0.135 s		—	
$u\bar{u} \to d\bar{d}$ QCD · 14 loop graphs	1	1 730	×6.9	251	fastest	2 650	×10.5	548	×2.2
	128	764	×3.0	254	fastest	2 670	×10.5	563	×2.2
	gen.	7.09 s		14.4 s		0.253 s		setup 0.0382 s	
$u\bar{u} \to t\bar{t}$ QCD · 14 loop graphs	1	2 010	×5.7	352	fastest	2 910	×8.3	655	×1.9
	128	745	×2.0	376	fastest	3 050	×8.1	686	×1.8
	gen.	6.39 s		17.1 s		1.12 s		setup 0.0103 s	
$u\bar{u} \to gg$ QCD · 45 loop graphs	1	6 830	×14.8	463	fastest	5 340	×11.5	1 700	×3.7
	128	3 100	×8.3	372	fastest	4 470	×12.0	1 700	×4.6
	gen.	21.2 s		29.1 s		0.183 s		setup 0.0149 s	
$gg \to gg$ QCD · 186 loop graphs	1	33 700	×13.0	2 590	fastest	36 100	×13.9	11 900	×4.6
	128	14 300	×6.9	2 070	fastest	33 200	×16.0	16 200	×7.8
	gen.	98.7 s		30.1 s		0.170 s		setup 0.0130 s	
$u\bar{u} \to ggg$ QCD · 556 loop graphs	1	194 000	×5.4	36 100	fastest	81 500	×2.3	36 900	×1.0
	128	125 000	×3.7	33 500	fastest	88 600	×2.6	43 400	×1.3
	gen.	863 s		23.9 s		0.724 s		setup 0.0109 s	
$e^- e^+ \to \mu^- \mu^+$ full EW, G_μ · 98 loop graphs	1	6 590	×2.1	validated, not timed		3 100	fastest	not timed	
	128	2 920	×1.0	validated, not timed		2 950	fastest	not timed	
	gen.	116 s		—		0.175 s		—	



The per-diagram design could not reach the $2 \to 5$ set: a seven-leg generation ran for more than nine hours without a result, and its counterterm enumeration alone needed up to 77 GiB. The v2 engine replaced its numerator, colour and counterterm core (Section 5) and keeps model import, the counterterm rules, Born weighting before reduction, the NinjaRS/oneloopmaster adapter, the exact carriers and the public interfaces.

7 Sources and reproducibility

- **Build:** `typst compile benchmarks.typ benchmarks.pdf` in `docs/performance_reports/loop/` (Typst 0.15).
- **Status, gates, in-flight work, decisions:** `data/milestone.json`, from the `Integrate …` merges on the first-parent history of e734b708 since 1df22af6 and the two commits after 45875b87; `workspace/reports/engine-v2/slice-status.md` (Current state, Integrated packages, e734b708 update, DECISIONS 2026-10-23); `HANDOFF_22_10_2026.md` §0–§3 (Rust: §0.4); MASTER_PLAN 2026-10-23 entries; `docs/development/loop_v2/REQUIREMENTS_ADDENDUM.md` A, D, F, G, H, J (I on `work/v2-poles-uvir`, d67ce126); DESIGN §6.1 on `work/v2-ct-recursion` (ac76825d, integrated into the slice at a18b7599).
- **Table 1:** provider columns HANDOFF §0.1 (addendum F); pyAmpliCol column DESIGN WP3.1 third increment "Generation per process" on `work/v2-generation-scale` (201efa8b). **Table 2:** DESIGN WP3.1 first to fourth increment, same branch (records `workspace/tmp/generation-scale/runs/`); the $gg \to gggg$ row from `cmp_gg_gggg.json`, `cmp_gg_gggg_p1_stab.json` and `cmp_gg_gggg_p1_dd.json` there. **Table 3:** DESIGN §1.3 Table 1b. **Conventions:** MadLoop standalone, Recola, OpenLoops 2.
- **Table 4, Table 5:** DESIGN §5.6 and §5.5 (data `workspace/reports/engine-v2/wp28-g1/`); the M/B ratios as §5.6 prints them.
- **Table 6** and NJet prose: `workspace/reports/engine-v2/njet-benchmark.md` §2, §5, §6 (raw `workspace/reference/runs/njet/timing/`); NJet, arXiv:1209.0100 Table 5.
- **Table 7, Table 8, first virtual:** `data/measured.json`, written by `scripts/collect_milestone.py <workspace>` from the records `gen_gg_ggggg_final`, `gen_gg_ggggg_hash`, `poles2_gg_ggggg`, `eval2_gg_ggggg`, `eval2_gg_ggggg_rot` and `cmp_uux_ggggg` in `workspace/tmp/generation-scale/runs/` and `gg_ggggg` in `workspace/tmp/ct-recursion/s4/` (JSON and `/usr/bin/time` logs); thread settings from DESIGN WP3.1 and ct-recursion DESIGN §6.1; the 119/3 shift and NJet deviations from njet-benchmark.md §5.3 and the fixture `tests/fixtures/loop/required/gg_ggggg_njet.json`; top-free values from DESIGN WP3.1 fourth increment and `tests/unit/test_loop_v2_topfree.py` (`work/v2-generation-scale`). Memory is shown in GiB = KiB/2^{20}.
- **Table 9, Section 5:** `docs/development/loop_v2/DESIGN.md` §1–§3, §5.5, §5.6, §6, §10, §12, REQUIREMENTS_ADDENDUM J (counterterm rule) and the integration records above.
- **Table 13:** `data/measurements.json`, `data/provider-cold.json` (legacy snapshot of 2026-10-18, unchanged) and `data/legacy-labels.json`; `data/colour-measurements.json` stays for the DESIGN citations. Removed at this milestone, in the git history at `45875b87:docs/performance_reports/loop/data/`: `processes.json`, `provider-audit.json` and `generation.json` (the 9.4 h and 77 GiB of Section 6); see the README.